8-Puzzle Problem

AIM: To write a Python program to solve the 8-Puzzle Problem using an appropriate search algorithm.

Algorithm:

1. Start the program.
2. Initialize the initial state and goal state of the 8-puzzle.
3. Represent the puzzle as a 3 × 3 matrix with one blank space.
4. Generate all possible moves by moving the blank space up, down, left, or right.
5. Apply a suitable search algorithm to explore the possible states.
6. Continue until the goal state is reached.
7. Display the solution path.
8. Stop the program.

Code :

```
import heapq

GOAL_STATE = (
    (1, 2, 3),
    (4, 5, 6),
    (7, 8, 0)
)

MOVES = [(-1, 0, 'UP'), (1, 0, 'DOWN'), (0, -1, 'LEFT'), (0, 1, 'RIGHT')]

def manhattan_distance(state):
    distance = 0
    goal_positions = {
        1: (0, 0), 2: (0, 1), 3: (0, 2),
        4: (1, 0), 5: (1, 1), 6: (1, 2),
        7: (2, 0), 8: (2, 1), 0: (2, 2)
    }
    for r in range(3):
        for c in range(3):
            val = state[r][c]
            if val != 0:
                gr, gc = goal_positions[val]
                distance += abs(r - gr) + abs(c - gc)
    return distance

def find_blank(state):
    for r in range(3):
        for c in range(3):
            if state[r][c] == 0:
                return r, c
    return None

def get_neighbors(state):
    neighbors = []
    r, c = find_blank(state)
    for dr, dc, move_name in MOVES:
        nr, nc = r + dr, c + dc
```

Output ;

```
PS D:\CSA1702-AI\Experiments> & C:\python311\python.exe d:/CSA1702-AI/Experiments/exp01_8_puzzle.py
Initial State:
1 2 3
4 _ 6
7 5 8

Goal State:
1 2 3
4 5 6
7 8 _

Solved in 2 steps:

Step 1: DOWN
1 2 3
4 5 6
7 _ 8

Step 2: RIGHT
1 2 3
4 5 6
7 8 _

PS D:\CSA1702-AI\Experiments>
```

RESULT:

Thus, the Python program to solve the 8-Puzzle Problem was executed successfully.